

Sprint 07 Full Structured Notes: Data Fetching, Server Actions, `useActionState`, `useOptimistic`, and Backend Integration

0. Where You Are Before Sprint 07

Before Sprint 07, you should already understand these React ideas:

- React builds user interfaces using components.
- JSX lets you write UI inside JavaScript.
- Props pass data from parent to child.
- `children` lets a component wrap other JSX.
- `map()` helps render lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `event.preventDefault()` stops a form from refreshing the page.
- `useState` lets React remember changing data.
- State updates should be immutable.
- Object state should be copied before changing fields.
- Array state should be updated with safe methods like `spread`, `filter()`, and `map()`.
- `useReducer` can organize related state actions.
- `useRef` can access DOM elements or store values that should not re-render the UI.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.

Sprints 1–6 mostly focused on **local frontend React skills**.

You learned how to build screens, split UI into components, pass data with props, handle user actions, manage state, run effects, clean up side effects, create custom hooks, and share state through Context.

Sprint 07 starts a new stage.

It teaches React how to work with **data outside React**.

That outside data usually comes from:

- an API,
- a backend server,
- a database through a backend,
- a form action,

- or a simulated async operation.

Simple version:

Sprint 07 teaches React how to talk to the outside world.

1. Sprint 07 Goal

The goal of Sprint 07 is to fetch data safely, handle async UI clearly, and organize beginner API/backend patterns.

By the end of this sprint, you should be able to:

- fetch data from an API,
- show loading UI while waiting,
- show data when the request succeeds,
- show an error message when the request fails,
- use `async` / `await` inside `useEffect` correctly,
- avoid making the `useEffect` callback directly `async`,
- compare Fetch API, Axios, and SWR at a beginner level,
- move API calls into `services/api.js`,
- use environment variables for API base URLs,
- create a beginner `useFetch` custom hook,
- understand the basic frontend-to-backend flow,
- understand the basic login/auth flow concept,
- understand what server actions are at a high level,
- use `useActionState` for action/form result state,
- use `useOptimistic` for instant UI feedback,
- use `startTransition` for lower-priority updates,
- build a small posts viewer,
- build a small optimistic task app.

Simple version:

Sprint 07 teaches you how to fetch data, handle waiting and errors, organize API code, and make async UI feel responsive.

2. Big Mental Model

A React app does not always have all the data immediately.

Sometimes React must ask another system for data.

Example:

```
User opens Posts page
↓
React renders the page
↓
The page starts fetching posts from an API
↓
While waiting, the UI shows Loading...
↓
If the request succeeds, the UI shows posts
↓
If the request fails, the UI shows an error message
```

This means async UI usually has three main states:

```
loading
success/data
error
```

Another way to think about it:

```
Request starts
├─ still waiting → loading UI
├─ succeeded → data UI
└─ failed → error UI
```

Beginner rule:

Never leave the user staring at a blank screen while async work is happening.

3. What Is Data Fetching?

Data fetching means asking another place for data.

That other place is usually an API.

Simple definition:

Data fetching means requesting data from an external source and using the response in your app.

Examples of fetched data:

- blog posts,
- products,
- user profiles,
- comments,
- tasks,
- weather data,
- search results,
- dashboard statistics,
- course lessons,
- notifications.

Example API request:

```
fetch("https://jsonplaceholder.typicode.com/posts")
```

This asks the API for posts.

React then uses the result to update the UI.

4. Data Fetching Is a Side Effect

In React, rendering should describe the UI.

Example rendering:

```
return <h1>Posts</h1>;
```

Fetching data is different.

Fetching data talks to something outside React.

That makes it a **side effect**.

Side effects include:

- fetching data,
- changing `document.title`,
- starting timers,
- adding event listeners,
- saving to local storage,
- connecting to a server.

Because fetching is a side effect, you usually run it inside `useEffect`.

Mental model:

```
React renders UI first
↓
useEffect runs after render
↓
fetch starts
↓
state updates when fetch finishes
↓
React re-renders with the result
```

Beginner rule:

Do not fetch directly during rendering. Fetch after render using `useEffect`.

5. The Three Main Async States

Most beginner fetch components need three pieces of state:

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

5.1 loading

`loading` means the request is still running.

Example UI:

```
if (loading) {
  return <p>Loading posts...</p>;
}
```

5.2 data

`data` means the request succeeded and returned useful information.

Example UI:

```
{posts.map((post) => (  
  <article key={post.id}>  
    <h2>{post.title}</h2>  
    <p>{post.body}</p>  
  </article>  
))}
```

5.3 error

`error` means something failed.

Example UI:

```
if (error) {  
  return <p>Error: {error}</p>;  
}
```

5.4 Summary Table

State	Meaning	UI
<code>loading</code>	Request is still running	Show loading message/spinner/skeleton
<code>data</code>	Request succeeded	Show the real content
<code>error</code>	Request failed	Show a helpful error message

Simple rule:

Async UI should always tell the user what is happening.

6. Basic Fetch API Pattern

The browser has a built-in function called `fetch()`.

Simple example:

```
const res = await fetch("https://jsonplaceholder.typicode.com/posts");  
const data = await res.json();
```

What this means:

```
fetch(url)
↓
ask the API for data
↓
wait for response
↓
convert response to JSON
↓
use the data
```

6.1 What Is JSON?

JSON is a common format for sending data between a backend and frontend.

Example JSON:

```
{
  "id": 1,
  "title": "Learn React",
  "completed": false
}
```

A frontend app often receives JSON from an API and turns it into UI.

7. Fetching Data With `useEffect`

Here is the correct beginner pattern.

```
import { useEffect, useState } from "react";

export default function PostsPage() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPosts() {
      try {
        setLoading(true);
        setError(null);

        const res = await fetch("https://jsonplaceholder.typicode.com/posts");
```

```

        if (!res.ok) {
            throw new Error("Failed to load posts");
        }

        const data = await res.json();
        setPosts(data.slice(0, 5));
    } catch (err) {
        setError(err.message);
    } finally {
        setLoading(false);
    }
}

loadPosts();
}, []);

if (loading) {
    return <p>Loading posts...</p>;
}

if (error) {
    return <p>Error: {error}</p>;
}

return (
    <main>
        <h1>Posts</h1>

        {posts.map((post) => (
            <article key={post.id}>
                <h2>{post.title}</h2>
                <p>{post.body}</p>
            </article>
        ))}
    </main>
);
}

```

8. Step-by-Step Explanation of the Fetch Example

8.1 Import Hooks

```
import { useEffect, useState } from "react";
```


You need:

- `useState` to store posts, loading, and error,
- `useEffect` to start the fetch after render.

8.2 Create State

```
const [posts, setPosts] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

Meaning:

- `posts` starts as an empty array,
- `loading` starts as `true` because the request begins when the page opens,
- `error` starts as `null` because there is no error yet.

8.3 Use `useEffect`

```
useEffect(() => {
  async function loadPosts() {
    // fetch logic
  }

  loadPosts();
}, []);
```

The empty dependency array means:

Run this effect once after the component first appears.

This is common for loading initial page data.

8.4 Use `try`, `catch`, and `finally`

```
try {
  // try to load data
} catch (err) {
  // handle failure
}
```

```
} finally {  
  // always run this  
}
```

Meaning:

Block	Job
<code>try</code>	Run the request
<code>catch</code>	Handle errors
<code>finally</code>	Stop loading whether success or failure

8.5 Check `res.ok`

```
if (!res.ok) {  
  throw new Error("Failed to load posts");  
}
```

`fetch()` does not always throw an error for bad HTTP responses.

So you should check `res.ok`.

If the response is not OK, you create an error yourself.

Beginner rule:

With Fetch API, check `res.ok` before using the response data.

8.6 Convert the Response to JSON

```
const data = await res.json();
```

This converts the API response into normal JavaScript data.

8.7 Update State

```
setPosts(data.slice(0, 5));
```

This stores the first 5 posts in React state.

When state changes, React re-renders the UI.

8.8 Show Loading UI

```
if (loading) {  
  return <p>Loading posts...</p>;  
}
```

If the request is still running, show loading UI.

8.9 Show Error UI

```
if (error) {  
  return <p>Error: {error}</p>;  
}
```

If the request failed, show the error.

8.10 Show Data UI

```
{posts.map((post) => (  
  <article key={post.id}>  
    <h2>{post.title}</h2>  
    <p>{post.body}</p>  
  </article>  
))}
```

If there is no loading and no error, show the posts.

9. Why `useEffect` Should Not Be Directly `async`

This is a common beginner mistake.

Wrong:

```
useEffect(async () => {  
  const res = await fetch("/api/posts");  
  const data = await res.json();  
  setPosts(data);  
}, []);
```

This looks reasonable, but it is not the correct React pattern.

Why?

An `async` function always returns a Promise.

React expects the `useEffect` callback to return either:

- nothing, or
- a cleanup function.

It should not return a Promise.

Correct:

```
useEffect(() => {  
  async function loadPosts() {  
    const res = await fetch("/api/posts");  
    const data = await res.json();  
    setPosts(data);  
  }  
  
  loadPosts();  
}, []);
```

Simple rule:

Do not make the `useEffect` callback `async`. Put an async function inside the effect and call it.

10. Why Loading and Error States Matter

Bad UI:

```
User opens page  
↓
```

Blank screen
↓
Eventually data appears

The user does not know what is happening.

Better UI:

User opens page
↓
Loading posts...
↓
Posts appear

Bad error experience:

Request fails
↓
Nothing happens

Better error experience:

Request fails
↓
Error: Failed to load posts

Real requests can fail because:

- the user is offline,
- the backend is down,
- the URL is wrong,
- the server returns an error,
- permission is denied,
- the response format is unexpected.

Beginner rule:

Every API request should have a loading plan and an error plan.

11. Fetch API vs Axios vs SWR

React does not force one data-fetching method.

Common options include:

- Fetch API,
- Axios,
- SWR,
- custom hooks,
- framework-specific data loading.

11.1 Fetch API

Fetch API is built into the browser.

Example:

```
const res = await fetch("/api/posts");
const data = await res.json();
```

Pros:

- no installation needed,
- built into modern browsers,
- good for learning fundamentals.

Beginner note:

- You must manually call `res.json()`.
- You should manually check `res.ok`.

11.2 Axios

Axios is a third-party HTTP library.

Example:

```
const response = await axios.get("/api/posts");
const data = response.data;
```

Pros:

- parses JSON automatically,
- has convenient error behavior,
- common in many React projects.

Beginner note:

- You must install it first.
 - It is not required for Sprint 07.
-

11.3 SWR

SWR is a data-fetching library.

It can help with:

- caching,
- refetching,
- loading state,
- error state,
- keeping server data fresh.

Beginner note:

- Useful later.
 - Do not start with SWR until you understand basic fetching.
-

11.4 Simple Comparison

Tool	What it is	Beginner note
Fetch API	Built-in browser API	Best starting point
Axios	Third-party HTTP library	Nice syntax, needs install
SWR	Data-fetching library	Useful later for server-state patterns

Beginner rule:

Learn Fetch API first. Then Axios and SWR will make more sense.

12. Why You Should Organize API Code

A beginner may write `fetch()` directly inside every component.

Example weak structure:

```
PostsPage.jsx has fetch()
ProfilePage.jsx has fetch()
Dashboard.jsx has fetch()
Login.jsx has fetch()
ProductsPage.jsx has fetch()
```

This becomes messy because:

- API URLs are repeated,
- error handling is repeated,
- components become harder to read,
- changing the API base URL becomes harder,
- backend communication is mixed with UI rendering.

Better structure:

```
src/
  services/
    api.js
  pages/
    PostsPage.jsx
```

Simple rule:

```
Component = UI logic
Service file = API communication
```

13. The `services/api.js` Pattern

Create this file:

```
src/services/api.js
```

Example:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

export async function getPosts() {
  const res = await fetch(`${API_BASE_URL}/posts`);
```



```
if (!res.ok) {  
  throw new Error("Failed to load posts");  
}  
  
return res.json();  
}
```

What this does:

- reads the base URL from an environment variable,
- calls the `/posts` endpoint,
- checks if the response is OK,
- throws an error if the request fails,
- returns JSON data if the request succeeds.

14. Environment Variables in Vite

Environment variables let you avoid hardcoding values everywhere.

Example `.env` file:

```
VITE_API_BASE_URL=https://jsonplaceholder.typicode.com
```

In Vite, frontend environment variables must start with:

```
VITE_
```

Then you can read it in code:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

Why this is useful:

- development API URL can be different from production API URL,
- you avoid repeating long URLs,
- API configuration stays in one place.

Important beginner warning:

Frontend environment variables are not secret. Do not put private passwords or secret API keys in Vite frontend `.env` files.

For Sprint 07, use `.env` for public configuration like the API base URL.

15. Using `getPosts()` From `services/api.js`

Example page:

```
import { useEffect, useState } from "react";
import { getPosts } from "../services/api";

export default function PostsPage() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPosts() {
      try {
        setLoading(true);
        setError(null);

        const data = await getPosts();
        setPosts(data.slice(0, 5));
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadPosts();
  }, []);

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <main>
      <h1>Posts</h1>

      {posts.map((post) => (
        <article key={post.id}>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </article>
      ))}
    </main>
  );
}
```

```
    </main>
  );
}
```

This page is cleaner because the API details are outside the component.

The component mainly handles:

- state,
- loading UI,
- error UI,
- rendering posts.

The service handles:

- API URL,
- fetch call,
- response check,
- JSON conversion.

16. Custom Hook: `useFetch`

You learned custom hooks in Sprint 06.

Sprint 07 uses the same idea for data fetching.

A `useFetch` hook lets you reuse loading/data/error logic.

Create:

```
src/hooks/useFetch.js
```

Code:

```
import { useEffect, useState } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadData() {
```

```
    try {
      setLoading(true);
      setError(null);

      const res = await fetch(url);

      if (!res.ok) {
        throw new Error("Request failed");
      }

      const json = await res.json();
      setData(json);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  loadData();
}, [url]);

return { data, loading, error };
}
```

17. How useFetch Works

17.1 The Hook Receives a URL

```
export function useFetch(url) {
```

The component gives the hook a URL.

Example:

```
useFetch("https://jsonplaceholder.typicode.com/posts")
```

17.2 The Hook Creates State

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

The hook manages the common async states.

17.3 The Hook Fetches Data in an Effect

```
useEffect(() => {
  async function loadData() {
    // fetch logic
  }

  loadData();
}, [url]);
```

The dependency is `[url]`.

This means:

```
Run after first render.
Run again if the URL changes.
```

17.4 The Hook Returns State

```
return { data, loading, error };
```

This gives the component everything it needs to render the correct UI.

18. Using `useFetch` in a Page

```
import { useFetch } from "../hooks/useFetch";

export default function PostsPage() {
  const {
    data: posts,
```

```

    loading,
    error,
  } = useFetch("https://jsonplaceholder.typicode.com/posts");

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <main>
      <h1>Posts</h1>

      {posts.slice(0, 5).map((post) => (
        <article key={post.id}>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </article>
      ))}
    </main>
  );
}

```

This line uses aliasing:

```
const { data: posts, loading, error } = useFetch(...);
```

It means:

Take data from the returned object,
but call it posts inside this component.

Beginner rule:

If the same fetching logic appears in multiple places, move it into a custom hook like `useFetch`.

19. Basic Backend Integration Bridge

Sprint 07 is not full MERN yet.

It is a beginner bridge between React and backend thinking.

The basic flow is:

```
React component
↓
calls service function
↓
service function calls backend/API
↓
backend returns data
↓
React stores result in state
↓
UI updates
```

Example:

```
PostsPage.jsx
↓
getPosts()
↓
GET /posts
↓
backend returns posts
↓
setPosts(posts)
↓
React shows post cards
```

20. Basic Auth Flow Concept

Authentication means checking who the user is.

A beginner login flow looks like this:

```
Login form
↓
POST /login
↓
backend checks username/password
↓
backend returns user/session/auth result
↓
frontend stores user/auth state
↓
```

```
protected routes check that state
↓
logout clears the state
```

In React terms:

```
Login form submits credentials
↓
authService.login(credentials)
↓
backend verifies credentials
↓
setUser(user)
↓
Navbar shows logged-in user
↓
Dashboard becomes accessible
```

Beginner warning:

Do not jump deeply into JWT, refresh tokens, cookies, and security architecture yet. First understand the basic flow.

Possible auth-related files later:

```
src/
  services/
    authService.js
  context/
    AuthContext.jsx
  pages/
    LoginPage.jsx
    DashboardPage.jsx
```

21. Server Actions: Beginner Concept

A server action is an action that runs on the server.

Simple definition:

A server action lets a form or event trigger logic on the server instead of only in the browser.

Example idea:

```
User submits form
↓
server action runs on server
↓
server saves data
↓
React receives result
↓
UI updates
```

Important beginner correction:

Server actions are not a normal plain Vite-only feature by themselves. They need a framework or runtime that supports them.

In a normal Vite React app, you can still learn the action-state pattern, but real server actions usually belong to framework environments such as React frameworks that support server features.

For Sprint 07, understand the concept lightly.

Do not go deep yet.

22. `useActionState`

`useActionState` is a React hook for managing state that changes because of an action.

It is commonly used with forms.

Simple definition:

`useActionState` lets an action return a new state, and React gives you the latest state plus a pending status.

Basic pattern:

```
const [state, formAction, isPending] = useActionState(actionFunction,
initialState);
```

Parts:

Part	Meaning
<code>state</code>	Current result state from the action
<code>formAction</code>	Function/action connected to the form
<code>isPending</code>	<code>true</code> while the action is running
<code>actionFunction</code>	Function that receives previous state and form data
<code>initialState</code>	Starting state before the action runs

23. `useActionState` Mental Model

```
User submits form
↓
Action function runs
↓
isPending becomes true
↓
Action returns result
↓
state updates
↓
isPending becomes false
```

This is useful because forms often need to show:

- submitting state,
- validation messages,
- success messages,
- error messages.

24. `useActionState` Example

```
import { useActionState } from "react";

async function submitFeedback(previousState, formData) {
  const message = formData.get("message");

  if (!message || message.trim() === "") {
    return {
      success: false,
```

```

        message: "Message is required.",
    };
}

await new Promise((resolve) => setTimeout(resolve, 1000));

return {
    success: true,
    message: "Feedback submitted.",
};
}

export default function FeedbackForm() {
    const [state, formAction, isPending] = useActionState(submitFeedback, {
        success: false,
        message: "",
    });

    return (
        <main>
            <h1>Feedback</h1>

            <form action={formAction}>
                <label htmlFor="message">Message</label>
                <textarea id="message" name="message" />

                <button type="submit" disabled={isPending}>
                    {isPending ? "Submitting..." : "Submit"}
                </button>
            </form>

            {state.message && <p>{state.message}</p>}
        </main>
    );
}

```

25. Step-by-Step Explanation of `useActionState`

25.1 The Action Function

```

async function submitFeedback(previousState, formData) {

```

The action receives:

- previous state,
 - form data.
-

25.2 Read Form Data

```
const message = formData.get("message");
```

This reads the value from:

```
<textarea id="message" name="message" />
```

Important:

`FormData` uses the input `name`, not the `id`.

25.3 Validate the Message

```
if (!message || message.trim() === "") {  
  return {  
    success: false,  
    message: "Message is required.",  
  };  
}
```

If the message is empty, return an error state.

25.4 Simulate Async Work

```
await new Promise((resolve) => setTimeout(resolve, 1000));
```

This waits one second.

It simulates a server request.

25.5 Return Success State

```
return {  
  success: true,  
  message: "Feedback submitted.",  
};
```

The returned object becomes the new `state`.

25.6 Use the Hook

```
const [state, formAction, isPending] = useActionState(submitFeedback, {  
  success: false,  
  message: "",  
});
```

This gives the component:

- current action result state,
 - a form action function,
 - pending status.
-

25.7 Connect the Form

```
<form action={formAction}>
```

This connects the form to the action.

25.8 Show Pending UI

```
<button type="submit" disabled={isPending}>  
  {isPending ? "Submitting..." : "Submit"}  
</button>
```

If the action is running:

- disable the button,
 - show "Submitting..."
-

25.9 Show Result Message

```
{state.message && <p>{state.message}</p>}
```

If there is a message, show it.

26. What Is `isPending`?

`isPending` tells you whether the action is currently running.

```
isPending = true  
↓  
Action is still running  
  
isPending = false  
↓  
Action is done
```

Common uses:

- disable submit button,
- show "Submitting...",
- prevent duplicate submissions,
- show temporary feedback.

Example:

```
<button disabled={isPending}>  
  {isPending ? "Saving..." : "Save"}  
</button>
```

Beginner rule:

Use `isPending` to show that the app is working after a form/action starts.

27. Optimistic UI

Normal async UI waits for the server before updating the screen.

Normal flow:

```
User adds task
↓
Show loading/saving
↓
Server confirms
↓
Task appears
```

Optimistic UI updates immediately.

Optimistic flow:

```
User adds task
↓
Task appears immediately as pending
↓
Server confirms
↓
Pending mark disappears
```

Simple definition:

Optimistic UI shows the expected result immediately before the server confirms it.

Why it is called optimistic:

The UI acts as if the server request will succeed.

28. When Optimistic UI Is Useful

Optimistic UI is useful for low-risk actions where success is likely.

Good examples:

- adding a task,
- liking a post,
- posting a short comment,
- starring an item,
- marking a notification as read.

Be careful with risky actions.

Avoid optimistic UI for high-risk actions like:

- payment,
- deleting an account,
- changing security settings,
- transferring money,
- submitting important official data.

Beginner rule:

Use optimistic UI for low-risk actions. Wait for real confirmation for high-risk actions.

29. `useOptimistic`

`useOptimistic` is a React hook that helps show optimistic UI.

Simple definition:

`useOptimistic` gives you a temporary version of state while async work is still happening.

Basic pattern:

```
const [optimisticState, addOptimisticValue] = useOptimistic(
  realState,
  updateFunction
);
```

Parts:

Part	Meaning
<code>realState</code>	Confirmed state
<code>optimisticState</code>	State shown in the UI, including temporary optimistic changes
<code>addOptimisticValue</code>	Function used to add an optimistic update
<code>updateFunction</code>	Function that describes how to build the optimistic state

30. `useOptimistic` Mental Model

Real tasks:
[Task A]


```
User adds Task B
↓
Optimistic tasks:
[Task A, Task B (Saving...)]

Server confirms
↓
Real tasks:
[Task A, Task B]
```

The user sees immediate feedback.

The app later reconciles with the real result.

31. Quick Tasks Example With `useOptimistic`

```
import { startTransition, useOptimistic, useState } from "react";

async function fakeSaveTask(text) {
  await new Promise((resolve) => setTimeout(resolve, 2000));

  return {
    id: crypto.randomUUID(),
    text,
  };
}

export default function QuickTasks() {
  const [tasks, setTasks] = useState([
    { id: "t1", text: "Learn useEffect" },
  ]);

  const [optimisticTasks, addOptimisticTask] = useOptimistic(
    tasks,
    (currentTasks, text) => [
      ...currentTasks,
      {
        id: crypto.randomUUID(),
        text,
        pending: true,
      },
    ]
  );
}
```

```

async function handleSubmit(event) {
  event.preventDefault();

  const formData = new FormData(event.currentTarget);
  const text = formData.get("task").trim();

  if (!text) return;

  event.currentTarget.reset();

  startTransition(() => {
    addOptimisticTask(text);
  });

  const savedTask = await fakeSaveTask(text);

  setTasks((prevTasks) => [...prevTasks, savedTask]);
}

return (
  <main>
    <h1>Quick Tasks</h1>

    <form onSubmit={handleSubmit}>
      <label htmlFor="task">Task</label>
      <input id="task" name="task" />
      <button type="submit">Add task</button>
    </form>

    <ul>
      {optimisticTasks.map((task) => (
        <li key={task.id}>
          {task.text} {task.pending && <span>Saving...</span>}
        </li>
      ))}
    </ul>
  </main>
);
}

```

32. Step-by-Step Explanation of Quick Tasks

32.1 Real Confirmed State

```
const [tasks, setTasks] = useState([
  { id: "t1", text: "Learn useEffect" },
]);
```

`tasks` stores confirmed tasks.

These are real tasks that the app trusts.

32.2 Optimistic State

```
const [optimisticTasks, addOptimisticTask] = useOptimistic(
  tasks,
  (currentTasks, text) => [
    ...currentTasks,
    {
      id: crypto.randomUUID(),
      text,
      pending: true,
    },
  ],
);
```

This says:

Start with real tasks.
When a user adds text optimistically,
show a temporary task with pending: true.

32.3 Handle Submit

```
async function handleSubmit(event) {
  event.preventDefault();
```

This stops the page from refreshing.

32.4 Read Form Data

```
const formData = new FormData(event.currentTarget);  
const text = formData.get("task").trim();
```

This reads the value from:

```
<input id="task" name="task" />
```

Again, `FormData` uses `name`.

32.5 Validate Empty Input

```
if (!text) return;
```

If the task is empty, stop.

32.6 Reset the Form

```
event.currentTarget.reset();
```

This clears the uncontrolled form input.

32.7 Add Optimistic Task

```
startTransition(() => {  
  addOptimisticTask(text);  
});
```

The task appears immediately with `Saving...`.

32.8 Wait for the Fake Save

```
const savedTask = await fakeSaveTask(text);
```

This simulates a 2-second backend save.

32.9 Store the Confirmed Task

```
setTasks((prevTasks) => [...prevTasks, savedTask]);
```

After the fake backend confirms, the task becomes real confirmed state.

33. `startTransition`

`startTransition` marks state updates as lower priority.

Simple definition:

`startTransition` tells React that an update is not urgent, so React can keep important UI interactions responsive.

Basic pattern:

```
startTransition(() => {  
  // lower-priority state update  
});
```

In Sprint 07, you mostly see it with optimistic UI:

```
startTransition(() => {  
  addOptimisticTask(text);  
});
```

34. Urgent vs Non-Urgent Updates

Some updates must feel immediate.

Urgent examples:

- typing into an input,
- clicking a button,
- checking a checkbox,
- focusing an input.

Some updates can be lower priority.

Non-urgent examples:

- rendering a large filtered list,
- switching a heavy tab,
- showing optimistic result,
- updating a complex dashboard section.

Simple comparison:

Update type	Example	Priority
Urgent	User typing	High
Non-urgent	Large UI recalculation	Lower

Beginner rule:

Do not use `startTransition` everywhere. Use it when an update can safely be treated as less urgent.

35. Normal Loading UI vs Optimistic UI

Normal loading UI:

```
User submits
↓
Show Saving...
↓
Wait for server
↓
Show result
```

Optimistic UI:

```
User submits
↓
Show expected result immediately
↓
Mark it as pending
↓
Server confirms
↓
Remove pending mark
```

Comparison:

Pattern	User sees result immediately?	Best for
Normal loading	No	Risky or important actions
Optimistic UI	Yes	Low-risk actions likely to succeed

36. Handling Failed Optimistic Updates

The example above shows the happy path.

But real optimistic UI should also think about failure.

Possible failure flow:

```
User adds task
↓
Task appears immediately
↓
Server fails
↓
Remove task or show failed status
```

Possible strategies:

1. Remove the optimistic item.
2. Mark the item as failed.
3. Show a retry button.
4. Show an error message.

Beginner version:

```
If save fails, show an error and do not add the task to real state.
```

For Sprint 07, you mainly need to understand the concept.

37. Common Sprint 07 Mistakes

Mistake 1: Making `useEffect` **Directly Async**

Wrong:

```
useEffect(async () => {  
  const res = await fetch("/api/posts");  
}, []);
```

Correct:

```
useEffect(() => {  
  async function loadPosts() {  
    const res = await fetch("/api/posts");  
  }  
  
  loadPosts();  
}, []);
```

Rule:

Put the async function inside the effect.

Mistake 2: No Error Handling

Weak:

```
const res = await fetch(url);  
const data = await res.json();  
setData(data);
```

Better:

```
const res = await fetch(url);  
  
if (!res.ok) {  
  throw new Error("Failed to load data");  
}  
  
const data = await res.json();  
setData(data);
```

Rule:

Always plan for failed requests.

Mistake 3: Blank Screen While Loading

Weak:

```
return <PostList posts={posts} />;
```

Better:

```
if (loading) return <p>Loading...</p>;  
if (error) return <p>Error: {error}</p>;  
  
return <PostList posts={posts} />;
```

Rule:

Show loading and error UI before showing data UI.

Mistake 4: Hardcoding API URLs Everywhere

Weak:

```
fetch("https://api.example.com/posts");  
fetch("https://api.example.com/users");  
fetch("https://api.example.com/products");
```

Better:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

Rule:

Put API configuration in one place.

Mistake 5: Scattering Fetch Calls Through Components

Weak:

Every component has its own API logic.

Better:

```
services/api.js handles API calls.  
Components focus on UI.
```

Rule:

Keep API communication in `services/` when the project grows.

Mistake 6: Forgetting `name` on Form Fields

Wrong:

```
<input id="task" />
```

Then this will not work properly:

```
formData.get("task")
```

Correct:

```
<input id="task" name="task" />
```

Rule:

`FormData` reads form fields by `name`.

Mistake 7: Using Optimistic UI for Risky Actions

Risky:

```
Show payment success before payment is confirmed.
```

Better:

```
Wait for payment confirmation.
```

Rule:

Use optimistic UI only when it is safe to assume success.

Mistake 8: Overusing `startTransition`

Wrong idea:

Wrap every state update in `startTransition`.

Better idea:

Use `startTransition` only for lower-priority updates.

Rule:

Normal state updates are fine most of the time.

38. Full Practice Project 1: Posts Viewer

Goal

Build a page that fetches posts and shows loading, error, and success UI.

Requirements

- Use `useEffect`.
- Use `useState`.
- Fetch posts from an API.
- Show `Loading posts...` while waiting.
- Show an error message if the request fails.
- Show the first 5 posts if the request succeeds.
- Use stable keys.
- Use semantic HTML.

Example Solution

```
import { useEffect, useState } from "react";

export default function PostsViewer() {
  const [posts, setPosts] = useState([]);
```

```

const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  async function loadPosts() {
    try {
      setLoading(true);
      setError(null);

      const res = await fetch("https://jsonplaceholder.typicode.com/posts");

      if (!res.ok) {
        throw new Error("Failed to load posts");
      }

      const data = await res.json();
      setPosts(data.slice(0, 5));
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  loadPosts();
}, []);

if (loading) {
  return <p>Loading posts...</p>;
}

if (error) {
  return <p>Error: {error}</p>;
}

return (
  <main>
    <h1>Posts Viewer</h1>

    <section aria-labelledby="posts-heading">
      <h2 id="posts-heading">Latest Posts</h2>

      {posts.map((post) => (
        <article key={post.id}>
          <h3>{post.title}</h3>
          <p>{post.body}</p>
        </article>
      ))}
    </section>
  </main>
);

```

```
    </section>
  </main>
);
}
```

39. Full Practice Project 2: Refactor to `services/api.js`

Goal

Move the API function out of the component.

Required Structure

```
src/
  services/
    api.js
  pages/
    PostsPage.jsx
```

`.env`

```
VITE_API_BASE_URL=https://jsonplaceholder.typicode.com
```

`src/services/api.js`

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

export async function getPosts() {
  const res = await fetch(`${API_BASE_URL}/posts`);

  if (!res.ok) {
    throw new Error("Failed to load posts");
  }

  return res.json();
}
```

`src/pages/PostsPage.jsx`

```
import { useEffect, useState } from "react";
import { getPosts } from "../services/api";
```

```

export default function PostsPage() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPosts() {
      try {
        setLoading(true);
        setError(null);

        const data = await getPosts();
        setPosts(data.slice(0, 5));
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadPosts();
  }, []);

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <main>
      <h1>Posts Page</h1>

      {posts.map((post) => (
        <article key={post.id}>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </article>
      ))}
    </main>
  );
}

```

40. Full Practice Project 3: useFetch Hook

Goal

Create reusable data-fetching logic.

Required Structure

```
src/  
  hooks/  
    useFetch.js  
  pages/  
    PostsPage.jsx
```

src/hooks/useFetch.js

```
import { useEffect, useState } from "react";  
  
export function useFetch(url) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    async function loadData() {  
      try {  
        setLoading(true);  
        setError(null);  
  
        const res = await fetch(url);  
  
        if (!res.ok) {  
          throw new Error("Request failed");  
        }  
  
        const json = await res.json();  
        setData(json);  
      } catch (err) {  
        setError(err.message);  
      } finally {  
        setLoading(false);  
      }  
    }  
  
    loadData();  
  });  
}
```

```
    }, [url]);

    return { data, loading, error };
  }
}
```

src/pages/PostsPage.jsx

```
import { useFetch } from "../hooks/useFetch";

export default function PostsPage() {
  const {
    data: posts,
    loading,
    error,
  } = useFetch("https://jsonplaceholder.typicode.com/posts");

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <main>
      <h1>Posts</h1>

      {posts.slice(0, 5).map((post) => (
        <article key={post.id}>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </article>
      ))}
    </main>
  );
}
```

41. Full Practice Project 4: Quick Tasks With Optimistic UI

Goal

Build a task app that shows a new task immediately while simulating a server save.

Requirements

- Use `useState` for real confirmed tasks.
- Use `useOptimistic` for temporary optimistic tasks.
- Use `startTransition` when adding the optimistic task.

- Simulate a 2-second save.
- Show `Saving...` next to pending tasks.
- Use a real form.
- Use a connected label.
- Use `event.preventDefault()`.

Example Solution

```
import { startTransition, useOptimistic, useState } from "react";

async function fakeSaveTask(text) {
  await new Promise((resolve) => setTimeout(resolve, 2000));

  return {
    id: crypto.randomUUID(),
    text,
  };
}

export default function QuickTasks() {
  const [tasks, setTasks] = useState([
    { id: "t1", text: "Learn data fetching" },
  ]);

  const [optimisticTasks, addOptimisticTask] = useOptimistic(
    tasks,
    (currentTasks, text) => [
      ...currentTasks,
      {
        id: crypto.randomUUID(),
        text,
        pending: true,
      },
    ],
  );

  async function handleSubmit(event) {
    event.preventDefault();

    const formData = new FormData(event.currentTarget);
    const text = formData.get("task").trim();

    if (!text) return;

    event.currentTarget.reset();

    startTransition(() => {
```

```

    addOptimisticTask(text);
  });

  const savedTask = await fakeSaveTask(text);

  setTasks((prevTasks) => [...prevTasks, savedTask]);
}

return (
  <main>
    <h1>Quick Tasks</h1>

    <form onSubmit={handleSubmit}>
      <label htmlFor="task">Task</label>
      <input id="task" name="task" />
      <button type="submit">Add task</button>
    </form>

    <section aria-labelledby="tasks-heading">
      <h2 id="tasks-heading">Tasks</h2>

      <ul>
        {optimisticTasks.map((task) => (
          <li key={task.id}>
            {task.text} {task.pending && <span>Saving...</span>}
          </li>
        ))}
      </ul>
    </section>
  </main>
);
}

```

42. Debug Task From Sprint 07

Broken Code

```

useEffect(async () => {
  const res = await fetch("/api/posts");
  const data = await res.json();
  setData(data);
}, []);

```

Problems

Problem 1:

```
useEffect(async () => {
```

The `useEffect` callback should not directly be `async`.

Problem 2:

```
const res = await fetch("/api/posts");  
const data = await res.json();
```

There is no `res.ok` check.

Problem 3:

```
setData(data);
```

There is no loading state or error state shown.

Fixed Version

```
useEffect(() => {  
  async function loadPosts() {  
    try {  
      setLoading(true);  
      setError(null);  
  
      const res = await fetch("/api/posts");  
  
      if (!res.ok) {  
        throw new Error("Failed to load posts");  
      }  
  
      const data = await res.json();  
      setData(data);  
    } catch (err) {  
      setError(err.message);  
    } finally {  
      setLoading(false);  
    }  
  }  
}
```

```
loadPosts();  
}, []);
```

What is fixed:

- async function moved inside `useEffect`,
- error handling added,
- loading state handled,
- failed responses handled with `res.ok`.

43. Sprint 07 Revision Questions

Question 1: Why do we keep loading and error state when fetching?

Because the request may take time or fail.

Loading state tells the user the app is working.

Error state tells the user something went wrong.

Without these states, the user may see a blank screen or silent failure.

Question 2: Why should `useEffect` not be directly marked `async`?

Because an async function returns a Promise.

React expects the effect callback to return either nothing or a cleanup function.

Correct pattern:

```
useEffect(() => {  
  async function loadData() {  
    // async work  
  }  
  
  loadData();  
}, []);
```

Question 3: What does `isPending` mean in `useActionState`?

`isPending` means the action is currently running.

Use it to:

- disable a submit button,
 - show "Submitting...",
 - prevent duplicate submissions,
 - show temporary feedback.
-

Question 4: How is optimistic UI different from a normal loading spinner?

A normal loading spinner waits before showing the result.

Optimistic UI shows the expected result immediately, then confirms it later.

Normal loading:

Submit → wait → show result

Optimistic UI:

Submit → show result immediately → confirm later

Question 5: Why is a `services/api.js` file better than scattering fetch calls through many components?

Because it keeps API logic organized in one place.

Benefits:

- components stay focused on UI,
 - API URLs are not repeated everywhere,
 - error handling can be reused,
 - code is easier to maintain,
 - backend changes are easier to handle.
-

Question 6: What is the difference between confirmed state and optimistic state?

Confirmed state is real data that has been accepted by the app or backend.

Optimistic state is temporary UI data shown before the backend confirms.

Example:

```
tasks = confirmed tasks
optimisticTasks = confirmed tasks + temporary pending tasks
```

Question 7: When should you use `startTransition` ?

Use it when an update can be lower priority.

Examples:

- optimistic update,
- rendering a heavy UI section,
- updating a large filtered list.

Do not use it for every state update.

Question 8: What belongs in `services/api.js` ?

API communication functions belong there.

Examples:

```
getPosts()
getUser()
loginUser()
createTask()
deletePost()
```

Components should call these service functions instead of containing all API details directly.

44. Sprint 07 Key Vocabulary

Data Fetching

Requesting data from an external source such as an API.

API

A system that lets the frontend request or send data.

Backend

The server-side part of an app that handles data, business logic, authentication, and database communication.

Request

A message sent from the frontend to an API/backend.

Response

The data or result sent back from the API/backend.

Loading State

State that tells the UI a request is still running.

Error State

State that stores request failure information.

Data State

State that stores successful response data.

Fetch API

The browser's built-in tool for making HTTP requests.

Axios

A third-party library for making HTTP requests.

SWR

A data-fetching library that helps with caching and keeping server data fresh.

```
res.ok
```

A property that tells whether the HTTP response was successful.

JSON

A common data format used between frontend and backend.

Environment Variable

A configurable value stored outside normal source code.

```
import.meta.env
```

How Vite exposes environment variables to frontend code.

```
services/
```

A folder commonly used for API/backend communication functions.

Custom Hook

A reusable function that starts with `use` and can call React hooks inside it.

```
useFetch
```

A custom hook commonly used to reuse data-fetching logic.

Server Action

An action that runs on the server in frameworks/runtimes that support server features.

```
useActionState
```

A hook for managing state returned by an action, often a form action.

```
isPending
```

A boolean that tells whether an action is still running.

Optimistic UI

UI that shows the expected result immediately before the server confirms it.

```
useOptimistic
```

A hook for showing temporary optimistic state while async work completes.

`startTransition`

A function that marks updates as lower priority.

45. Sprint 07 Mental Models

Mental Model 1: Fetching Data

```
Render page
↓
useEffect starts fetch
↓
show loading
↓
request succeeds or fails
↓
show data or error
```

Mental Model 2: API Organization

```
Component asks for data
↓
services/api.js handles request
↓
component receives result
↓
component renders UI
```

Mental Model 3: `useFetch`

```
Component gives URL to useFetch
↓
useFetch manages loading/data/error
↓
component uses returned values
↓
component renders correct UI
```

Mental Model 4: Auth Flow

```
Login form
↓
POST /login
↓
backend verifies user
↓
frontend stores user state
↓
protected route checks user
↓
logout clears user state
```

Mental Model 5: `useActionState`

```
Form submits
↓
action runs
↓
isPending true
↓
action returns result
↓
state updates
↓
isPending false
```

Mental Model 6: Optimistic UI

```
User does action
↓
UI shows expected result now
↓
server works in background
↓
UI confirms or handles failure
```

46. Sprint 07 Checklist

You understand Sprint 07 when you can do these without looking at notes:

- Explain why fetching data is a side effect.
 - Write loading, data, and error state.
 - Fetch data inside `useEffect` using an inner async function.
 - Explain why `useEffect(async () => {})` is wrong.
 - Check `res.ok` before using response data.
 - Show loading UI.
 - Show error UI.
 - Render fetched data with stable keys.
 - Explain Fetch API vs Axios vs SWR at a beginner level.
 - Create `services/api.js`.
 - Read `VITE_API_BASE_URL` from `.env`.
 - Move API calls out of components.
 - Create a basic `useFetch` hook.
 - Explain the login/auth flow concept.
 - Explain what server actions are at a high level.
 - Use `useActionState` for form result state.
 - Explain `isPending`.
 - Explain optimistic UI.
 - Use `useOptimistic` to show a pending task immediately.
 - Use `startTransition` for a lower-priority optimistic update.
 - Explain when optimistic UI is risky.
-

47. Mandatory Practice Coding Block

Warm-Up Drills

Do these from memory:

1. Write three state values for loading, data, and error.
 2. Write a `useEffect` with an inner async function.
 3. Write a `fetch()` request with `res.ok` checking.
 4. Write a loading return.
 5. Write an error return.
 6. Write `useOptimistic` syntax.
 7. Write `startTransition(() => {})` syntax.
 8. Write a `.env` variable named `VITE_API_BASE_URL`.
-

Guided Coding Task

Build a Posts Viewer.

Requirements:

- Fetch posts from an API.
 - Show loading state.
 - Show error state.
 - Show first 5 posts.
 - Use `article` for each post.
 - Use `key={post.id}`.
-

Refactor Task

Refactor the Posts Viewer.

Requirements:

- Create `src/services/api.js`.
 - Move `getPosts()` into it.
 - Create `.env`.
 - Use `VITE_API_BASE_URL`.
 - Import `getPosts()` into the page.
-

Custom Hook Task

Create `useFetch`.

Requirements:

- Create `src/hooks/useFetch.js`.
 - The hook receives a URL.
 - The hook returns `{ data, loading, error }`.
 - Use it in a page.
-

Optimistic UI Task

Build Quick Tasks.

Requirements:

- A form adds a task.
- The task appears immediately.
- The task shows `Saving...` while pending.
- A fake 2-second save confirms it.
- Use `useOptimistic`.

- Use `startTransition`.
-

Challenge Task

Improve Quick Tasks with failure handling.

Requirements:

- Randomly fail the fake save sometimes.
 - Show an error message if save fails.
 - Do not add failed tasks to confirmed state.
 - Optionally mark failed optimistic tasks as failed.
-

48. Expected Output / Self-Check

By the end of Sprint 07 practice:

- The Posts page does not show a blank screen while fetching.
 - Loading state appears before data loads.
 - Error state appears if the request fails.
 - Posts render after the request succeeds.
 - API logic is moved into `services/api.js`.
 - The base URL is read from `.env`.
 - `useFetch` can fetch from any URL.
 - The Quick Tasks app shows a task immediately.
 - Pending tasks show `Saving...`.
 - Confirmed tasks remain after save completes.
 - You can explain why optimistic UI feels faster.
 - You can explain when optimistic UI is unsafe.
-

49. Sprint 07 Definition of Done

You are done with Sprint 07 when you can explain and build this without notes:

```
React renders UI.
useEffect starts an API request after render.
The UI shows loading while waiting.
If the request succeeds, data is shown.
If the request fails, an error is shown.
API calls belong in services/.
Repeated fetch logic can become useFetch().
useActionState helps manage action/form result state.
```

```
isPending tells whether the action is still running.  
useOptimistic shows expected UI immediately before the server confirms.  
startTransition marks lower-priority updates.
```

Shortest correct mental model:

Sprint 07 teaches React how to fetch data, handle waiting, handle errors, organize API code, and make async UI feel responsive.

50. Sprint Retrospective Prompts

Answer these after completing the practice tasks:

1. What felt easy in Sprint 07?
2. What broke first?
3. Did loading, data, and error state make sense?
4. Did `useEffect` with an inner async function feel natural?
5. Did `services/api.js` make the component cleaner?
6. Did `useFetch` feel reusable?
7. Did optimistic UI feel different from normal loading UI?
8. What concept needs one more small app?
9. What would you refactor if this became a real project?
10. What still feels unclear about backend integration?